



Control de un Motor de Paso Mediante un Controlador Comercial

Un Proyecto de Sistema Embebido para Aplicaciones de Precisión

Ricardo Santana

Universidad Central de Venezuela, Facultad de Ingeniería, Escuela de Ingeniería Eléctrica

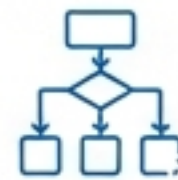
El Desafío de Ingeniería: Precisión, Control y Retroalimentación

Desarrollar una aplicación embebida que controle un motor de paso mediante un controlador comercial, en respuesta a la variación del ángulo de un encoder.

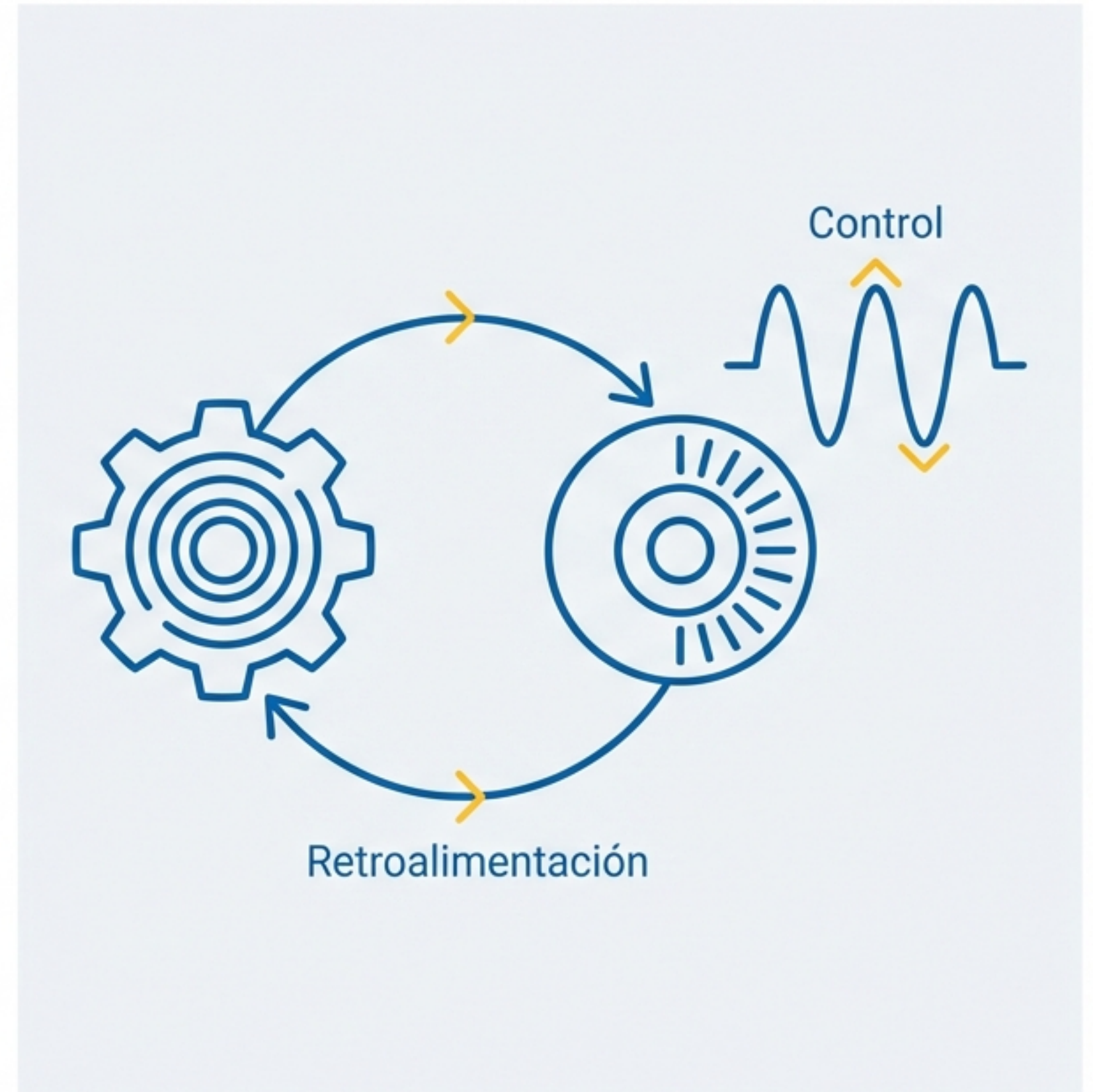
El sistema debe, además, **mostrar los parámetros clave** en una pantalla LCD mediante comunicación I2C.

Arquitectura de Software:

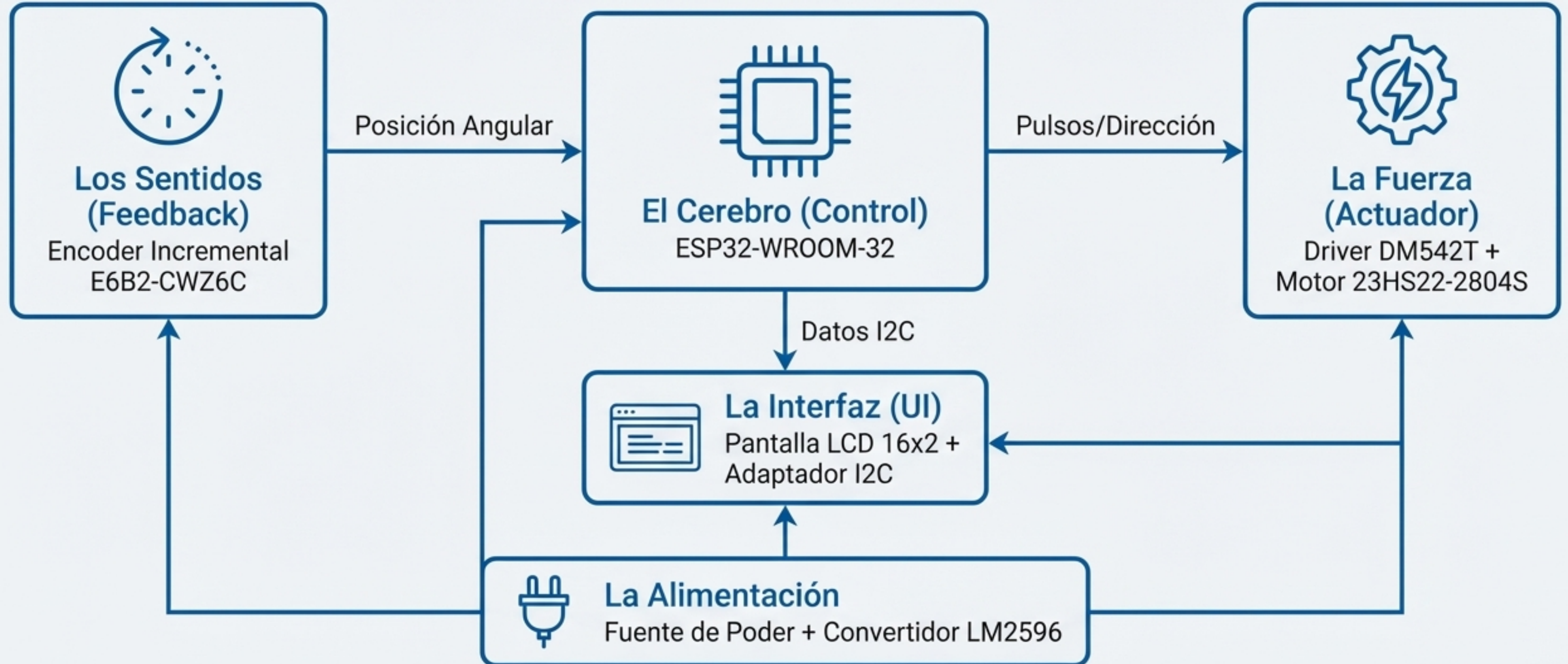
Roboto Regular – Modular y concurrente, desarrollada sobre el framework ESP-IDF.



Resultado Final: Un sistema completo con documentación técnica y software reutilizable.



Arquitectura del Sistema de Control de Movimiento



Componentes Clave del Ecosistema de Hardware

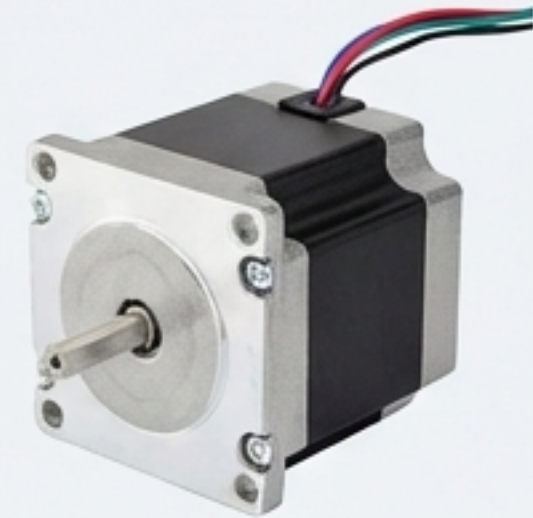
Los elementos esenciales para un control preciso y una interfaz de usuario eficaz.



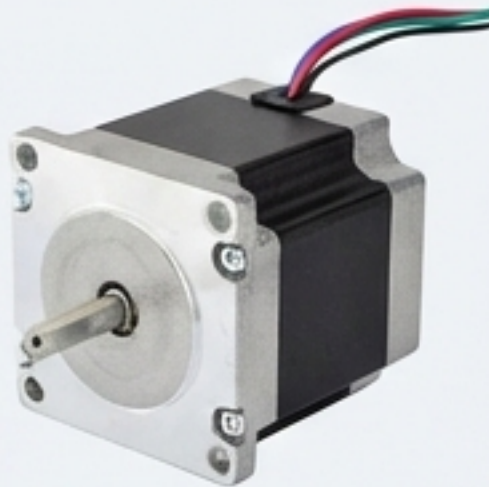
Microcontrolador:
ESP32-WROOM-32



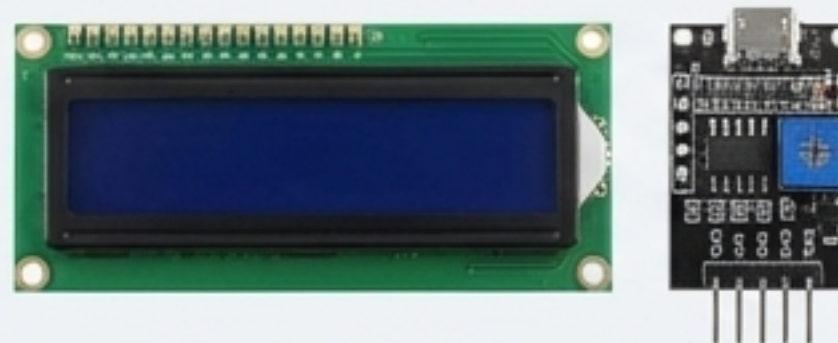
Driver de Motor: Digital Stepper
Drive DM542T



Motor de Paso: StepperOnline
23HS22-2804S



Encoder Rotativo: Omron
E6B2-CWZ6C



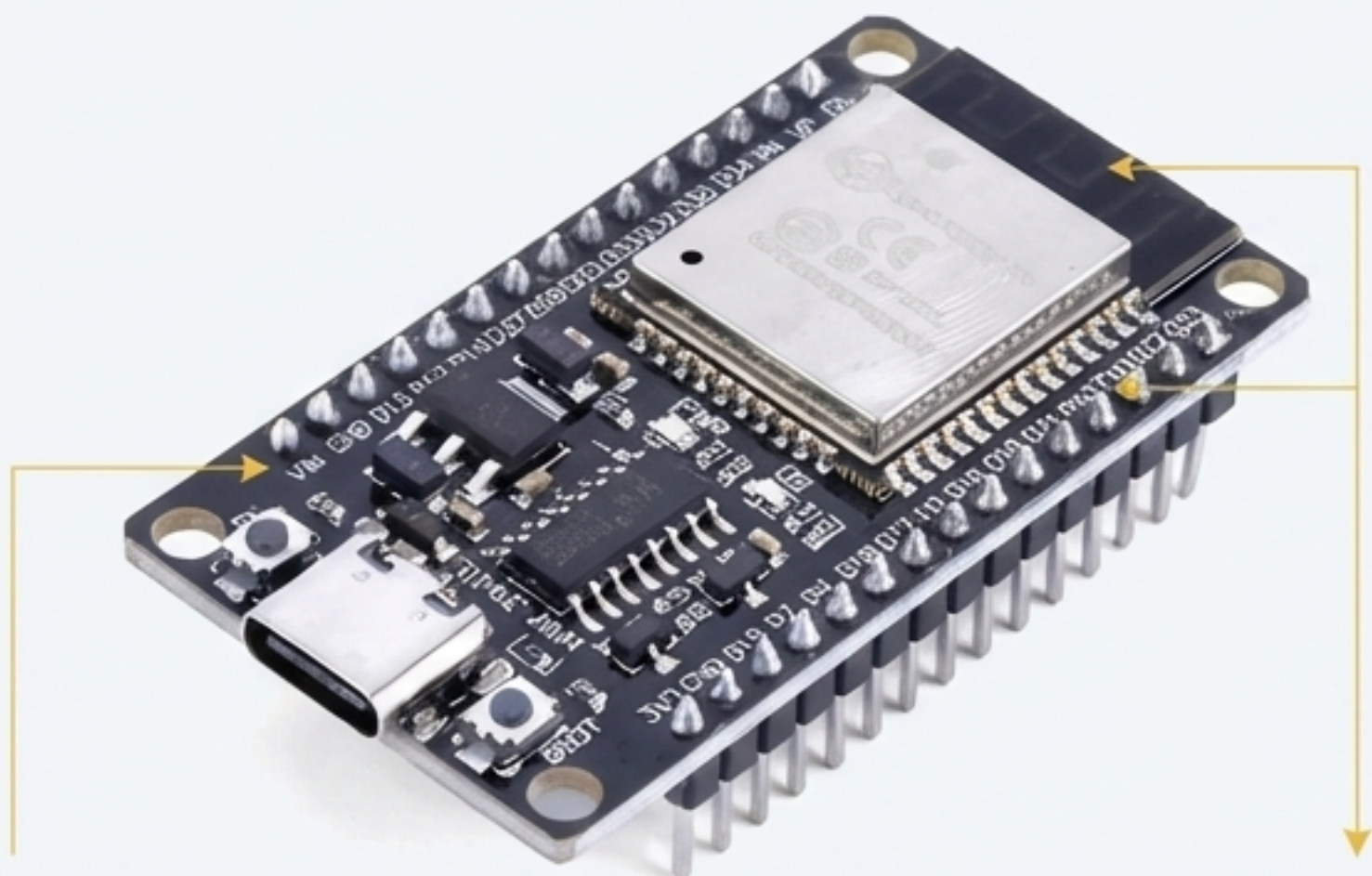
Pantalla: LCD WH1602B con
Adaptador I2C



Regulador de Voltaje: Módulo
LM2596

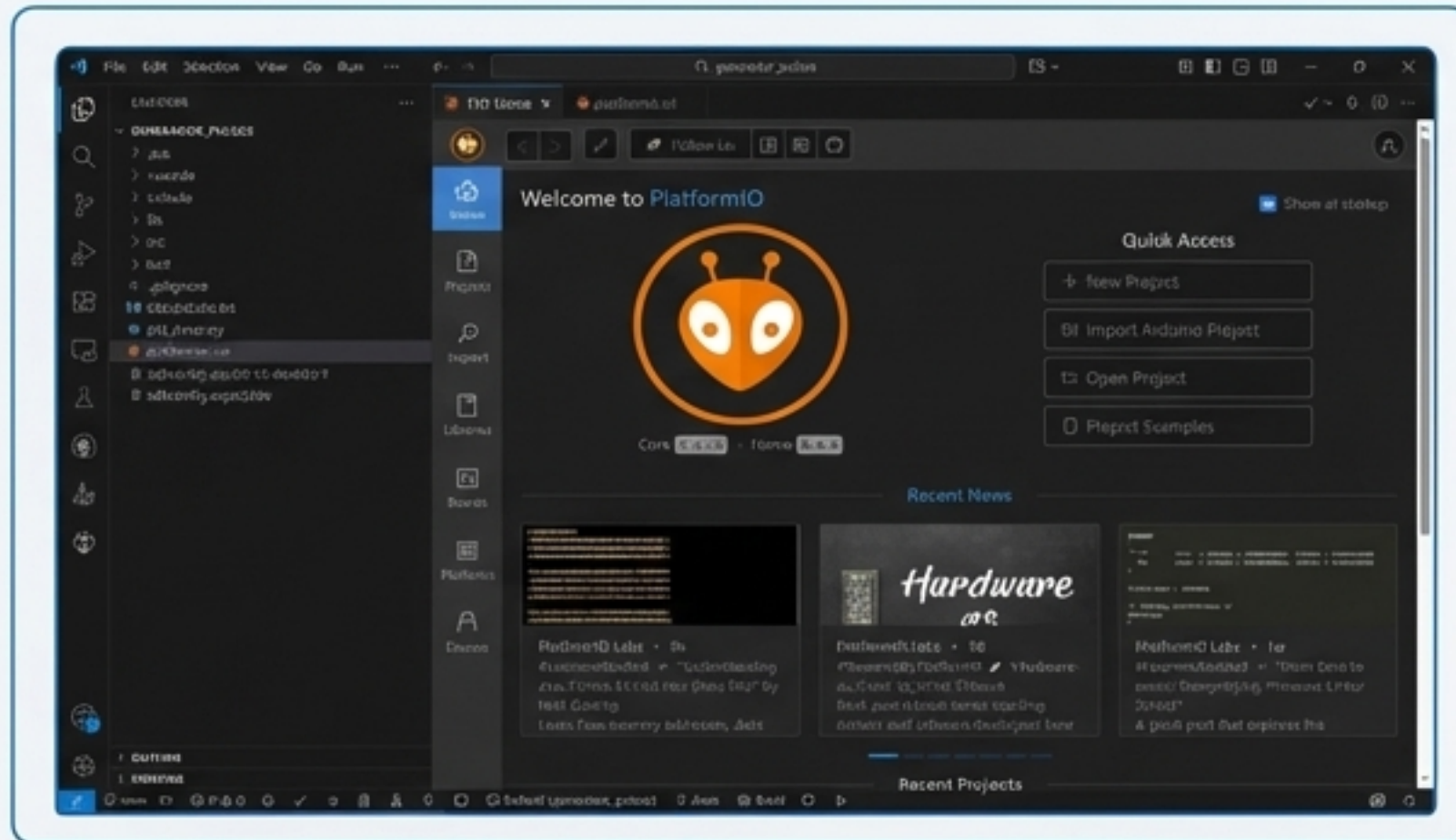
El Cerebro: El ESP32 y el Entorno de Desarrollo Profesional

ESP32-WROOM-32



- **Microprocesador** Xtensa® dual-core de 32 bits LX6 (hasta **240 MHz**).
- Ricos periféricos integrados, incluyendo **PCNT** (**Pulse Counter**) para lectura de encoder y **LEDC** (**PWM**) para control de motor.
- Conectividad **Wi-Fi y Bluetooth** para futuras aplicaciones IoT.

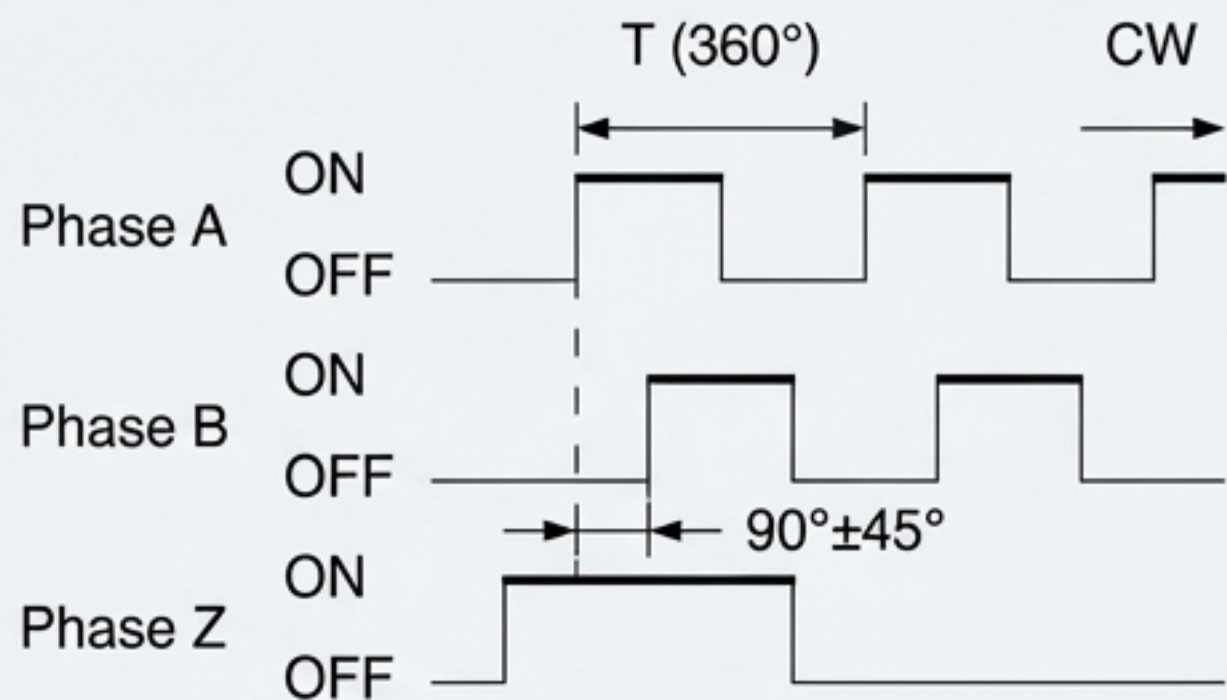
Entorno de Desarrollo



- **IDE:** Visual Studio Code con extensión PlatformIO.
- **Framework:** ESP-IDF de Espressif para control de bajo nivel y uso de FreeRTOS.
- **Metodología:** Desarrollo de librerías modulares y reutilizables.

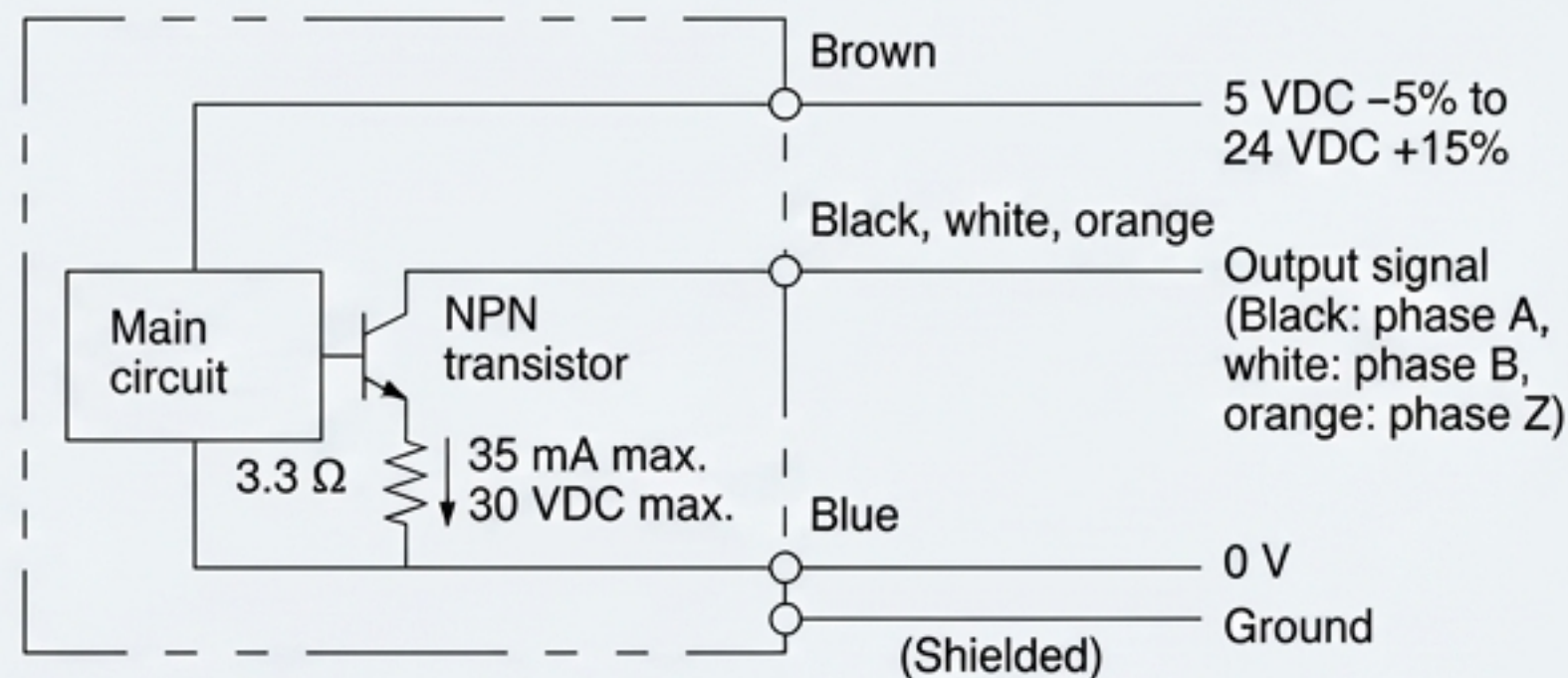
Los Sentidos: Caracterización del Encoder Incremental E6B2-CWZ6C

Principio de Funcionamiento



Las fases A y B entregan pulsos en cuadratura (desfase de 90°) para determinar posición y dirección. La fase Z genera un pulso de índice por cada revolución para una referencia absoluta.

Circuito de Salida y Conexión

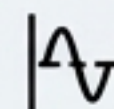


Salida de colector abierto NPN. Requiere resistencias pull-up externas o la activación de las internas del ESP32 para una correcta lectura de los niveles lógicos.

Especificaciones Clave

 **Resolución:** 1024 Pulsos por Revolución (P/R).

 **Voltaje de Operación:** 5 a 24 VDC.

 **Frecuencia Máxima:** 100 kHz.

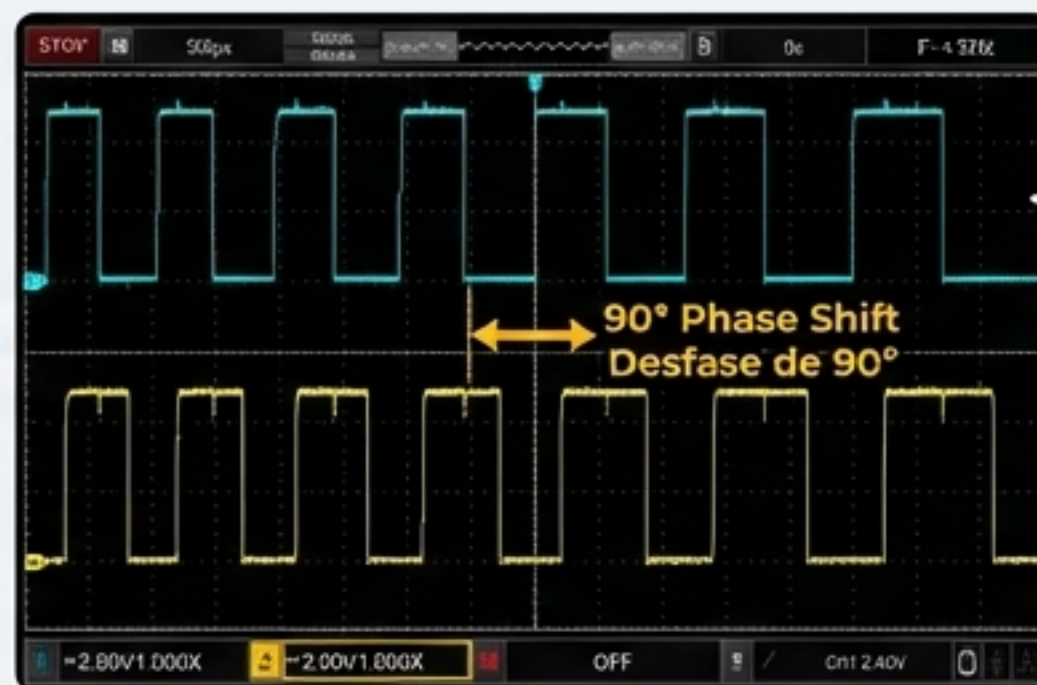
De la Señal Eléctrica al Dato: Validación e Integración del Encoder

Paso 1: Montaje de Prueba



Validación inicial del funcionamiento del encoder con alimentación de 5V y sondas de osciloscopio.

Paso 2: Señal Observada



Confirmación visual de las señales en cuadratura, validando la integridad del sensor.

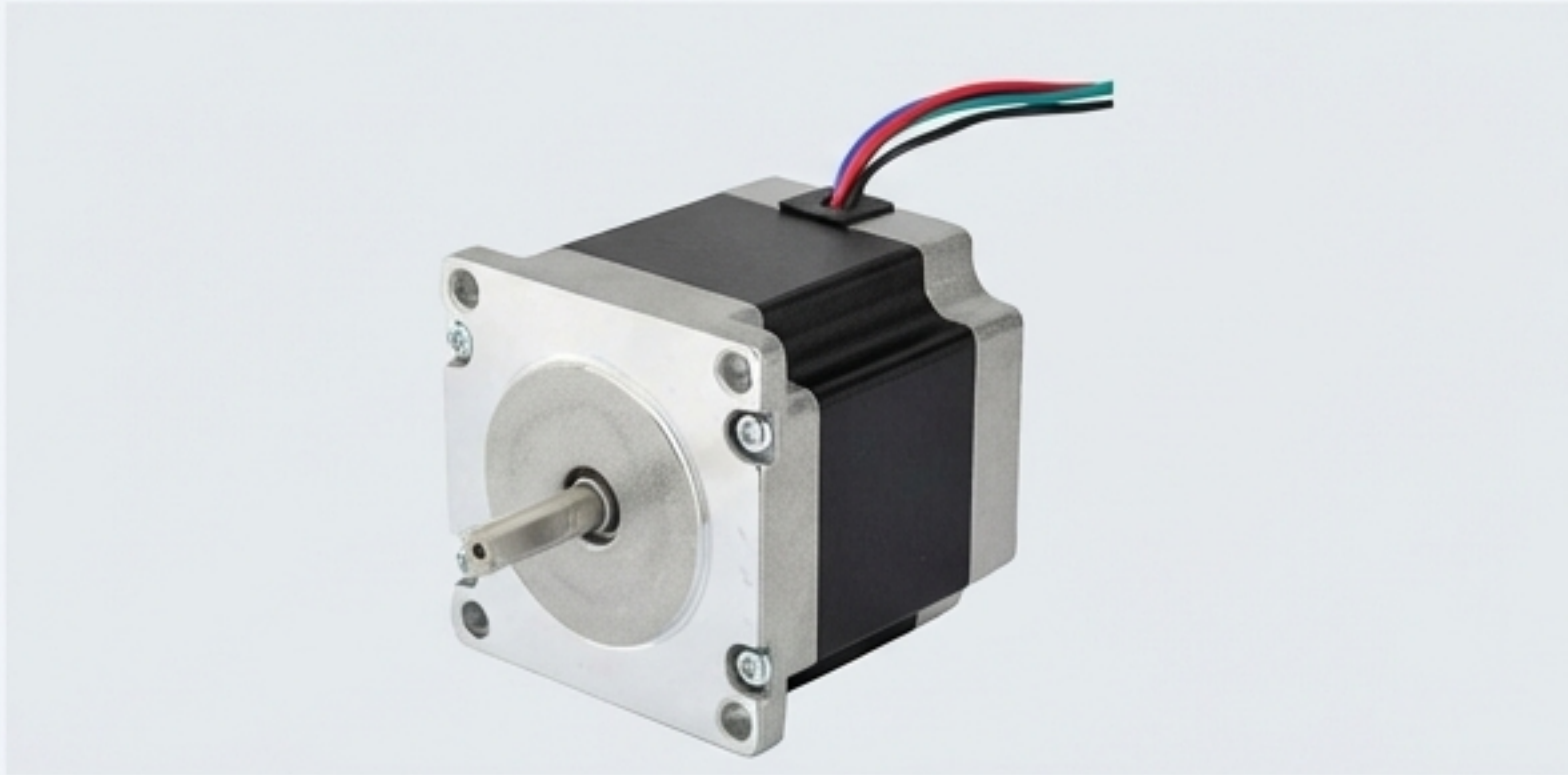
Paso 3: Integración con ESP32 y Salida de Datos

```
1 (266) msin_task: Calling geo_esin()<ES><LF>
1 (256) ENCODER_118: Librería Lirerda inicializada en pines A:27, 8:14, 2:12<CR><LF>
1 (766) RAIN_APP: Angulo del Encoder: 0.00 grados<CR><LF>
1 (766) RAIN_APP: Angulo del Encoder: 0.00 grados<CR><LF>
1 (1266) RAIN_APP: Angulo del Encoder: 27.45 grados<CR><LF>
1 (1280) RAIN_APP: Angulo del Encoder: 04.16 grados<CR><LF>
1 (2256) RAIN_APP: Angulo del Encoder: 33.22 grados<CR><LF>
1 (2756) RAIN_APP: 51Fase de 1 detectado! El contador se ha reseteado.<CR><LF>
1 (2226) RAIN_APP: Angulo del Encoder: 79.01 grados<CR><LF>
1 (3756) RAIN_APP: Angulo del Encoder: 25.26 grados<CR><LF>
1 (5290) RAIN_APP: Angulo del Encoder: 19.96 grados<CR><LF>
0 (3756) RAIN_APP: 51Pulse de 1 detectado! El contador se ha reseteado.<CR><LF>
1 (3766) RAIN_APP: Angulo del Encoder: -193.11 grados<CR><LF>
1 (3786) RAIN_APP: Angulo del Encoder: -237.33 grados<CR><LF>
1 (5786) RAIN_APP: Angulo del Encoder: -240.91 grados<CR><LF>
1 (6286) RAIN_APP: Angulo del Encoder: -109.16 grados<CR><LF>
1 (5766) RAIN_APP: Angulo del Encoder: 13.95 grados<CR><LF>
0 (7786) RAIN_APP: 11Pulse de 1 detectado! El contador se ha reseteado.<CR><LF>
1 (7780) RAIN_APP: Angulo del Encoder: 98.39 grados<CR><LF>
1 (8266) RAIN_APP: Angulo del Encoder: 58.15 grados<CR><LF>
1 (8766) RAIN_APP: Angulo del Encoder: 37.65 grados<CR><LF>
1 (5760) RAIN_APP: Angulo del Encoder: -12.78 grados<CR><LF>
0 (9750) RAIN_APP: 11Pulse de 1 detectado! El contador se ha reseteado.<CR><LF>
1 (10202) RAIN_APP: Angulo del Encoder: -47.00 grados<CR><LF>
1 (16756) RAIN_APP: Angulo del Encoder: -144.83 grados<CR><LF>
1 (11286) RAIN_APP: Angulo del Encoder: -267.01 grados<CR><LF>
```

El firmware del ESP32, usando el periférico PCNT, interpreta las señales para calcular el ángulo y detectar el pulso de índice (Z), reseteando el contador.

La Fuerza: Motor de Paso Híbrido y Driver Digital DM542T

Panel Izquierdo: El Motor (23HS22-2804S)



- **Tipo:** Híbrido Bipolar, 2 Fases.
- **Ángulo de Paso:** **1.8°**.
- **Corriente/Fase:** **2.80 A**.
- **Torque de Sujeción (Holding Torque):** **1.20 N-m**.

Panel Derecho: El Driver (DM542T)



- **Control:** Entradas ópticamente aisladas para Pulsos (**PUL**), Dirección (**DIR**) y Habilitación (**ENA**).
- **Potencia:** Voltaje de alimentación de **20-50 VDC**, soporta hasta **4.5 A**.
- **Precisión:** **16 resoluciones** de micropasos (hasta 25,600 pulsos/rev).
- **Inteligencia:** Tecnología **Anti-Resonancia** para un movimiento suave y bajo calentamiento.

Configuración Fina del Driver: Corriente y Resolución de Micropasos

Tabla de Configuración de Corriente

Peak Current	RMS Current	SW1	SW2	SW3
1.00A	0.71A	ON	ON	ON
1.46A	1.04A	OFF	ON	ON
1.91A	1.36A	ON	OFF	ON
2.37A	1.69A	OFF	OFF	ON
2.84A	2.03A	ON	ON	OFF
3.31A	2.36A	OFF	ON	OFF
3.76A	2.69A	ON	OFF	OFF
4.50A	3.20A	OFF	OFF	OFF
4.50A	3.30A	OFF	OFF	OFF

Para el motor 23HS22-2804S (2.8A), la configuración ideal es SW1=ON, SW2=ON, SW3=OFF (2.84A Peak).

Para una resolución de 3200 pasos/rev (microstep 16), la configuración es SW5=ON, SW6=ON, SW7=OFF, SW8=ON.

Tabla de Configuración de Micropasos

Microstep	Steps/rev. (for 1.8°motor)	SW5	SW6	SW7	SW8
1	200	ON	ON	ON	ON
2	400	OFF	ON	ON	ON
4	800	ON	OFF	ON	ON
8	1600	OFF	OFF	ON	ON
16	3200	ON	ON	OFF	ON
32	6400	OFF	ON	OFF	ON
64	12800	ON	OFF	OFF	ON
128	25600	OFF	OFF	OFF	ON
5	1000	ON	ON	ON	OFF
10	2000	OFF	ON	ON	OFF
20	4000	ON	OFF	ON	OFF
25	5000	OFF	OFF	ON	OFF
40	8000	ON	ON	OFF	OFF
50	10000	OFF	ON	OFF	OFF
100	20000	ON	OFF	OFF	OFF
125	25000	OFF	OFF	OFF	OFF

Nota Adicional: El switch SW4 permite reducir la corriente en reposo al 50% o 90% para minimizar el calentamiento del motor.

Del Código al Movimiento: Implementación de la Librería `stepperMotor`

Panel Izquierdo: La Abstracción del Software

Se desarrolló una librería (`stepperMotor.h` / `.c`) para abstraer el control de bajo nivel. Utiliza el periférico LEDC del ESP32 para generar trenes de pulsos precisos y no bloqueantes.

```
// Estructura de configuración para el usuario
typedef struct {
    gpio_num_t step_pin; // Pin para pulsos (PUL)
    gpio_num_t dir_pin;  // Pin para dirección (DIR)
} stepper_config_t;

// Función principal para mover el motor
esp_err_t stepper_move(int steps, int speed_hz, int
direction);
```

Panel Derecho: Ejemplo de Uso en `main.c`

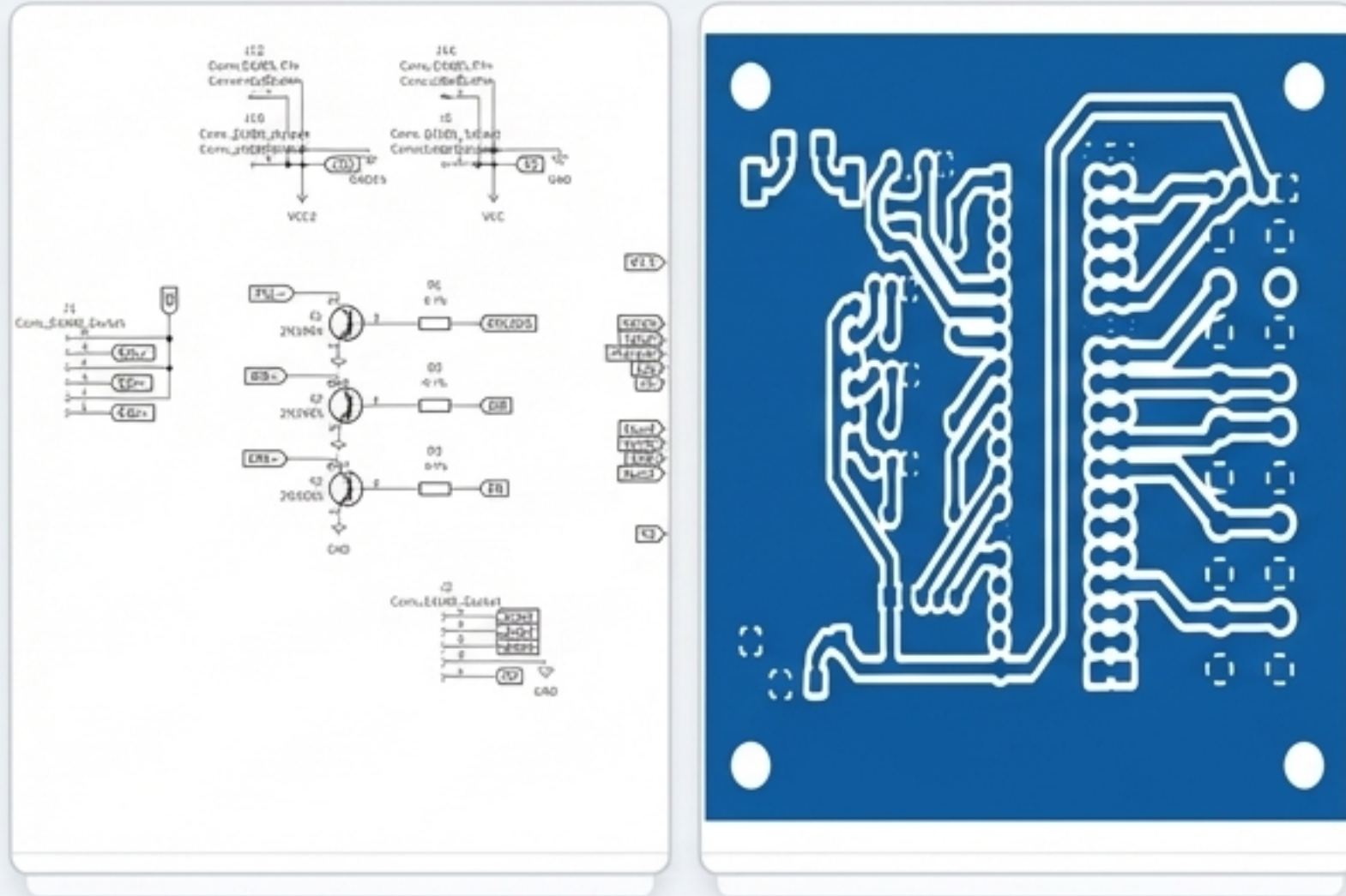
El uso de la librería en la aplicación principal es simple e intuitivo, permitiendo un control directo sobre el movimiento.

```
// Mover 10000 pasos a 10kHz en dirección 1
ESP_LOGI("MAIN", "Moviendo a la derecha...");
stepper_move(10000, 10000, 1);
vTaskDelay(pdMS_TO_TICKS(1000));

// Mover 20000 pasos a una velocidad mayor
ESP_LOGI("MAIN", "Moviendo a la izquierda más rápido...");
stepper_move(20000, 20000, 0); // Frecuencia de pulso
aumentada
```

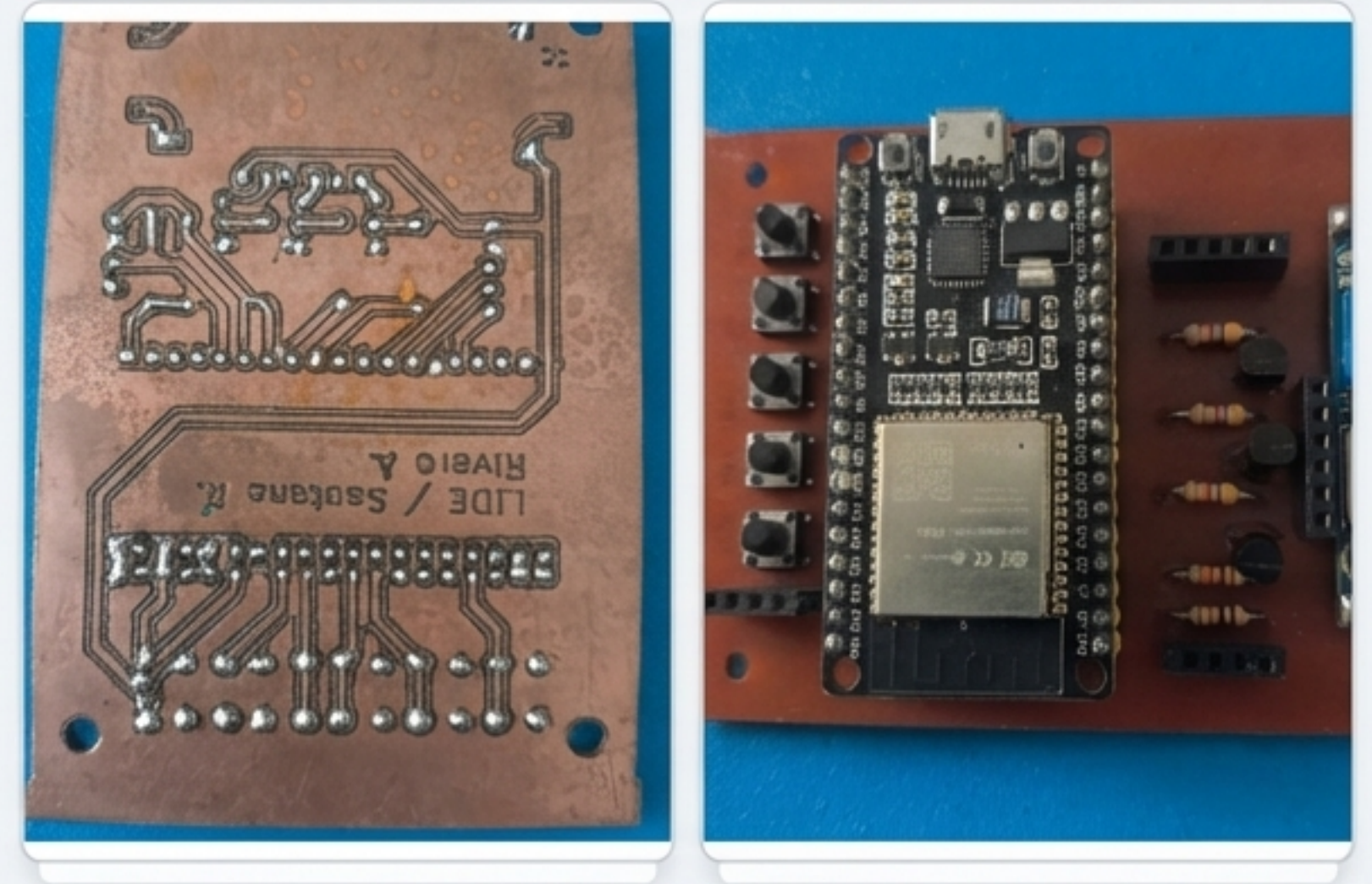

Del Prototipo al Módulo Final: Diseño y Fabricación de PCB

Sección 1: Diseño Esquemático y Layout



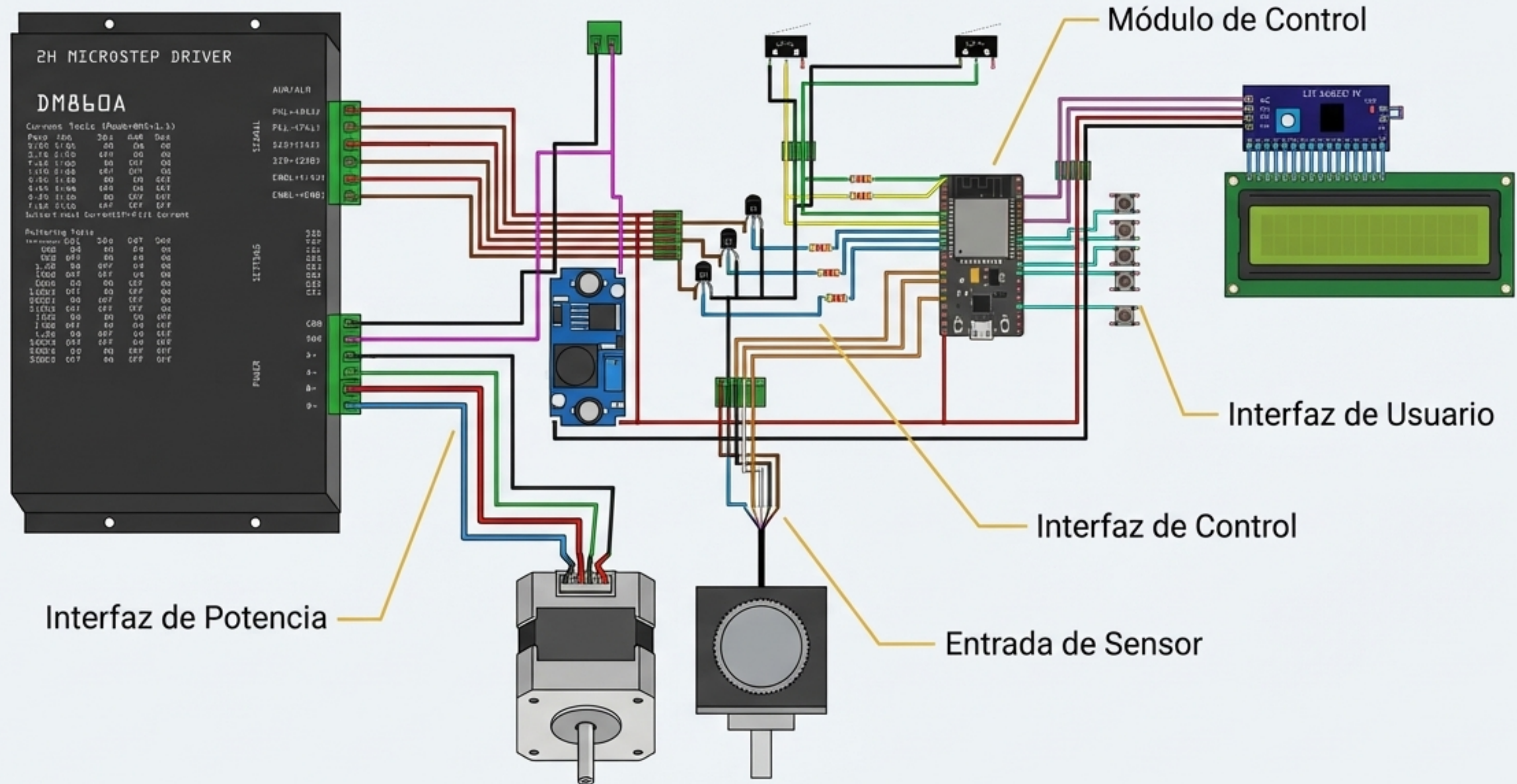
Diseño de un módulo de trabajo a medida para integrar el ESP32, conectores para el driver, encoder, LCD y botones, asegurando conexiones robustas y fiables.

Sección 2: Módulo Fabricado y Ensamblado



El resultado es un módulo de control compacto y funcional que sirve como la base para la aplicación final y futuras experimentaciones.

Diagrama de Conexiones del Sistema Integrado



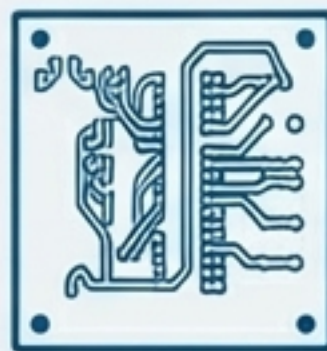
Conclusiones y Activos Reutilizables del Proyecto

Logros Clave



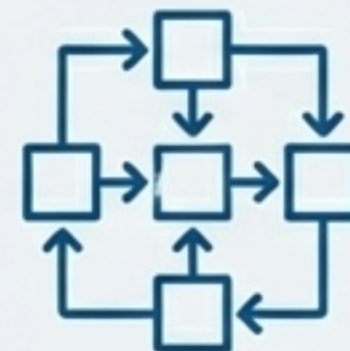
Software Robusto

Se crearon librerías modulares para ESP-IDF que permiten controlar el driver DM542T y leer encoders incrementales con el periférico PCNT. El código es reutilizable para cualquier proyecto que utilice la familia ESP32.



Hardware Validado

Se diseñó y fabricó un módulo de control basado en ESP32, que sirve como una plataforma de desarrollo estable y probada.



Integración Exitosa

Se demostró la integración completa de todos los componentes (control, potencia, sensado e interfaz) en un sistema funcional y coherente.

Impacto y Aplicaciones Potenciales

Las librerías y el hardware desarrollado son directamente aplicables en proyectos de robótica, máquinas CNC, impresoras 3D y otros sistemas de automatización que requieran control de movimiento de precisión.

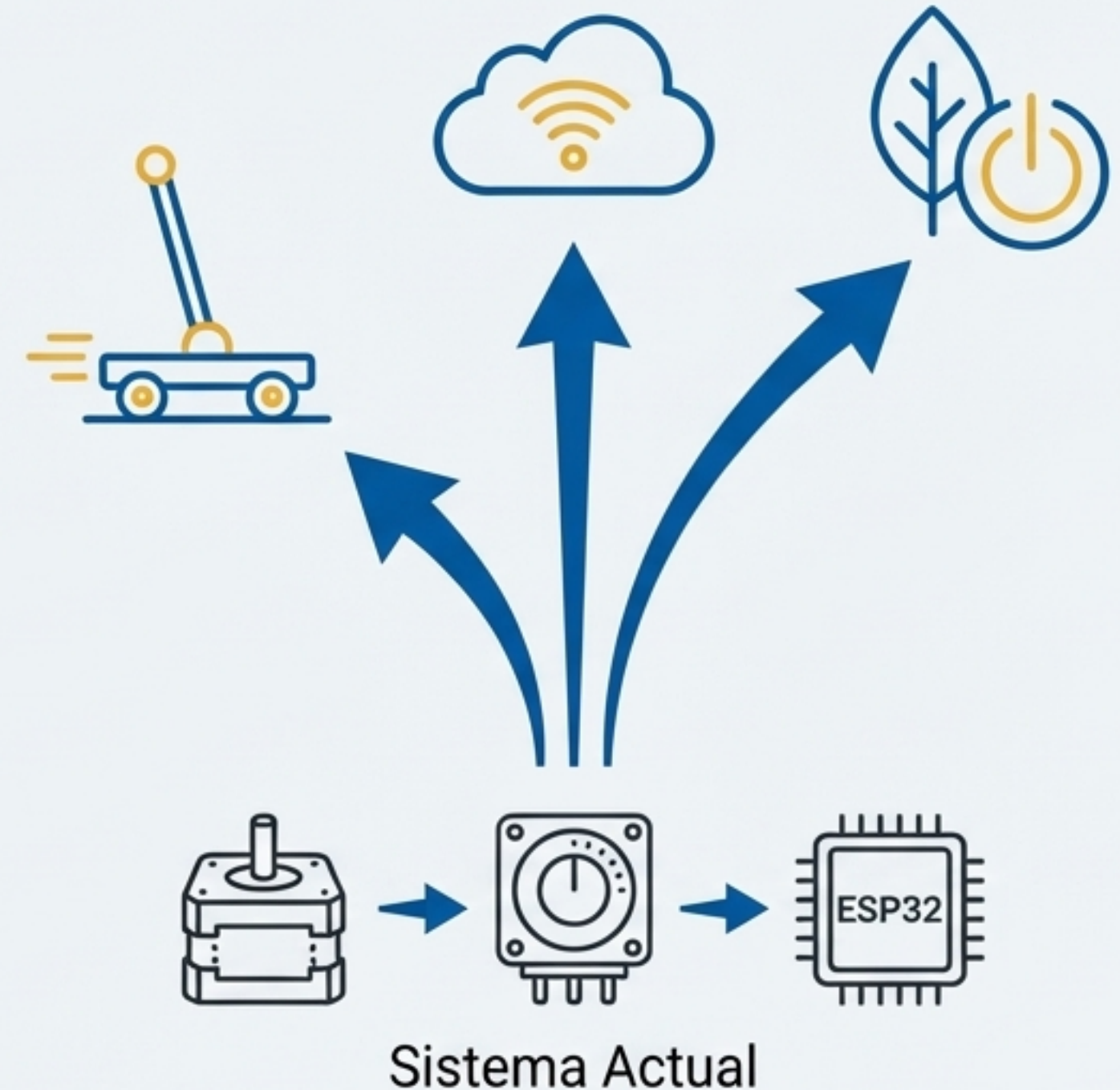
Trabajo Futuro y Evolución del Sistema

Próximos Pasos Técnicos

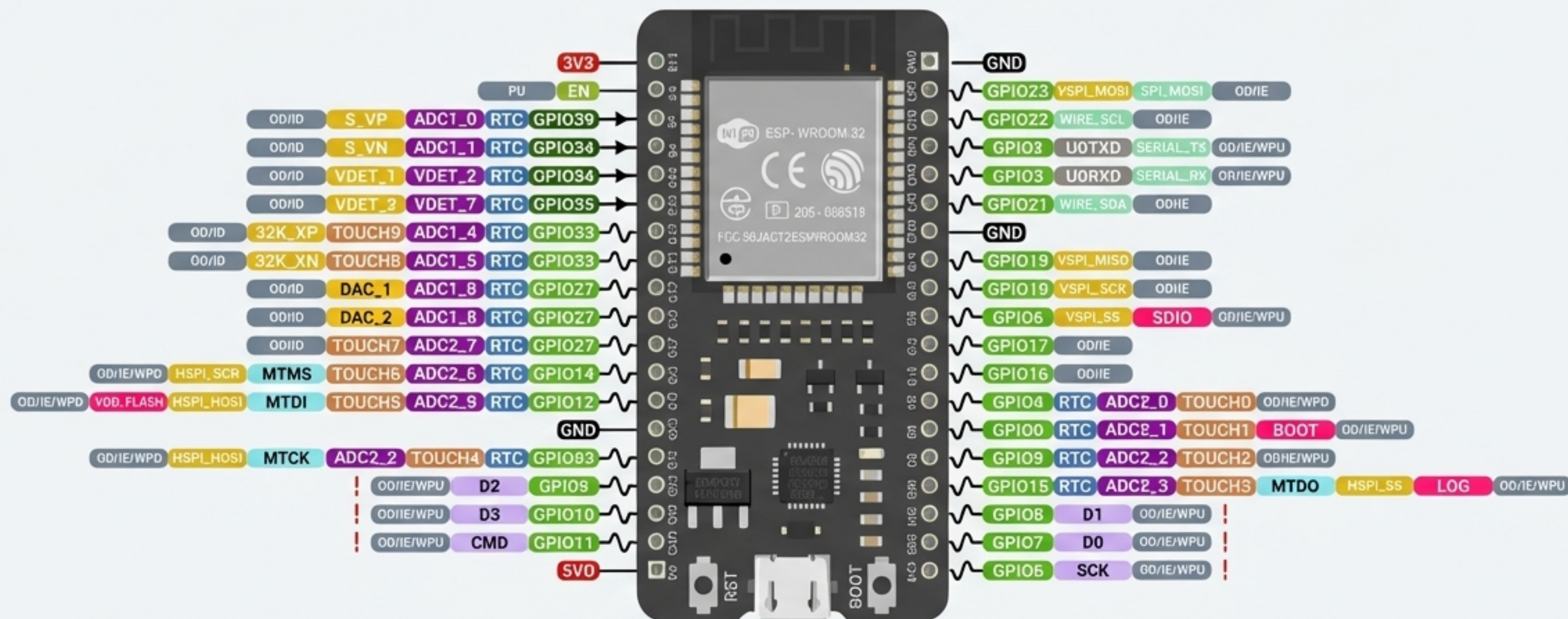
- **Control en Lazo Cerrado:** Implementar un sistema de control avanzado, como el de un péndulo invertido, utilizando la retroalimentación del encoder para cerrar el lazo de control en tiempo real.
- **Optimización:** Mejorar la gestión de memoria y el consumo energético del firmware, que no fueron un foco principal durante el desarrollo inicial.

Visión a Largo Plazo

- **Conectividad IoT:** Aprovechar las capacidades Wi-Fi y Bluetooth del ESP32 para desarrollar aplicaciones de monitoreo y control remoto, integrando el sistema en el ecosistema del Internet de las Cosas (IoT).



Anexo: Diagrama de Pines (Pinout) del ESP32-WROOM-32



Referencia detallada de los pines del módulo ESP32-WROOM-32 utilizado en el proyecto, mostrando las capacidades de cada GPIO (ADC, Touch, PWM, I2C, SPI, etc.).